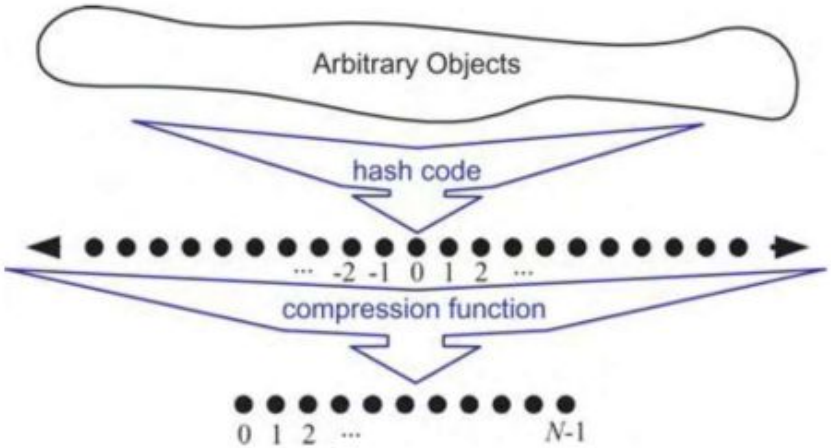
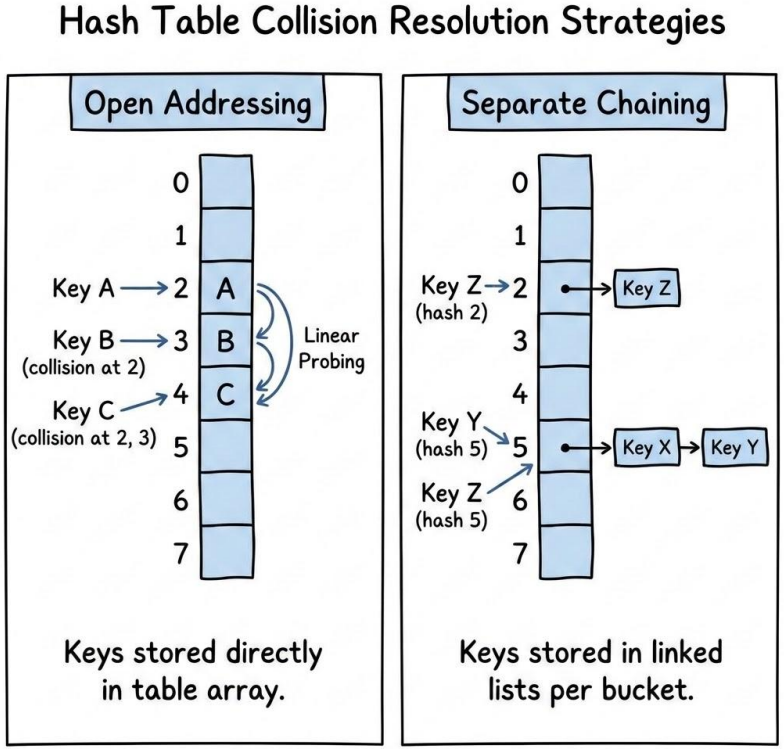
Estructuras de Datos

2026
Primer Cuatrimestre

REPASO


Arbitrary Objects
hash code
... -2 -1 0 1 2 ...
compression function
0 1 2 ... N-1


Hash Table Collision Resolution Strategies
Open Addressing
0
1
Key A → 2
A
Key B → 3
(collision at 2)
B
Linear Probing
Key C → 4
(collision at 2, 3)
C
5
6
7
Keys stored directly in table array.
Separate Chaining
0
1
Key Z → 2
(hash 2)
Key Z
3
4
Key Y
(hash 5)
5
Key X
Key Y
Key Z
(hash 5)
6
7
Keys stored in linked lists per bucket.

TIEMPO DE EJECUCIÓN

Pregunta

¿Cómo podemos evaluar la calidad de nuestros algoritmos e implementaciones?

Eficiencia

- Capacidad del software de ejecutarse usando la mínima cantidad de recursos de hardware
  - CPU
  - Memoria
- Objetivo: Una aplicación debe buscar ser lo más eficiente posible

Tiempo de Ejecución

¿Cómo podemos evaluar la calidad de nuestros algoritmos e implementaciones?

Dados dos algoritmos, o implementaciones, que hacen lo mismo pero de manera diferente ¿cómo podemos decidir cuál es mejor que el otro?

Tiempo de Ejecución

push de TDA Pila implementado con TDA Cola

```
public E push(E e) {
    Queue<E> auxiliar = new LinkedList<>();
    while(!this.queue.isEmpty()) {
        auxiliar.add(this.queue.poll());
    }
    this.queue.add(e);
    while(!auxiliar.isEmpty()) {
        this.queue.add(auxiliar.poll());
    }
    return e;
}
```

push de TDA Pila implementado con Nodo enlazado

```
public E push(E e) {
    Node<E> topeViejo = this.tope;
    this.tope = new Node<E>(e, topeViejo);
    this.cantidad++;
    return e;
}
```

Tiempo de Ejecución

Vamos a tomar como un factor de comparación, y como un factor de evaluación de calidad, al análisis del tiempo de ejecución.

Tiempo de Ejecución

Vamos a tomar como un factor de comparación, y como un factor de evaluación de calidad, al análisis del tiempo de ejecución.

Cómo podemos responder la pregunta *“¿Cuánto tiempo tarda tu programa?”*

Tiempo de Ejecución

Vamos a tomar como un factor de comparación, y como un factor de evaluación de calidad, al análisis del tiempo de ejecución.

Cómo podemos responder la pregunta *“¿Cuánto tiempo tarda tu programa?”*

Bajo esta pregunta, estamos buscando la solución que ejecute en menos tiempo.

Tiempo de Ejecución

Propuesta 1: Calculemos el tiempo, en milisegundos, que tarda una operación en resolverse

Tiempo de Ejecución

Propuesta 1: Calculemos el tiempo, en milisegundos, que tarda una operación en resolverse

```
public static int cantidadDeElementos(LinkedList<?> lista) {
    int contador = 0;
    for (Object elemento : lista) {
        contador++;
    }
    return contador;
}
```

Tiempo de Ejecución

Propuesta 1: Calculemos el tiempo, en milisegundos, que tarda una operación en resolverse

```java
public static LinkedList<Integer> generateRandomLinkedList(int x) {
    // Crear una LinkedList para almacenar los enteros aleatorios
    LinkedList<Integer> randomList = new LinkedList<>();

    // Crear un objeto Random para generar números aleatorios
    Random random = new Random();

    // Generar x enteros aleatorios y agregarlos a la LinkedList
    for (int i = 0; i < x; i++) {
        int randomInt = random.nextInt(); // Número aleatorio de 32 bits
        randomList.add(randomInt);
    }

    return randomList;
}
```

Tiempo de Ejecución

Propuesta 1: Calculemos el tiempo, en milisegundos, que tarda una operación en resolverse

```java
public static void main(String[] args) {

    for( int i = 1; i< 10; i++ ) {
        LinkedList<Integer> lista = generateRandomLinkedList(5000000*i);

       // Registra el tiempo de inicio
       long inicio = System.currentTimeMillis();

       // Llama al método que deseas medir
       int cantidad = cantidadDeElementos(lista);

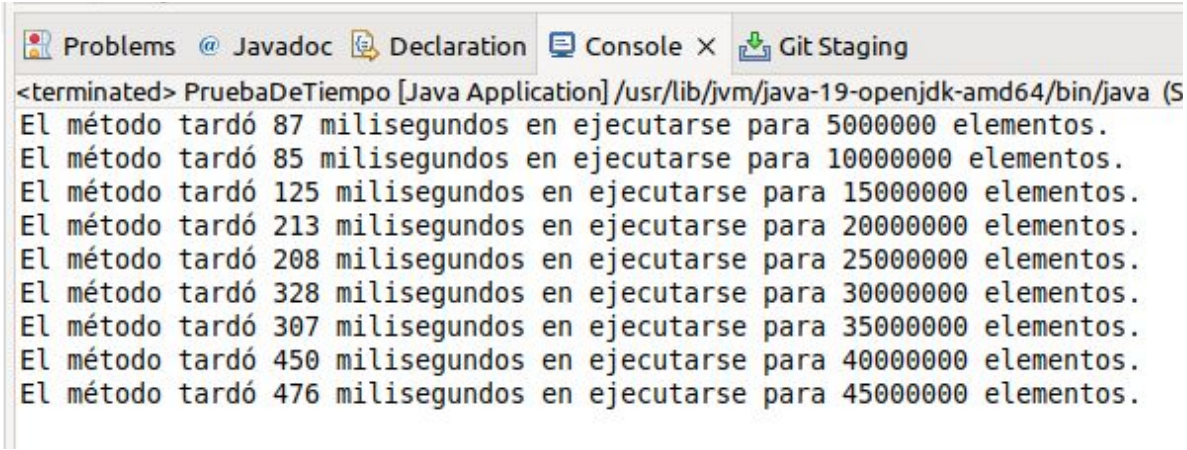
       // Registra el tiempo de finalización
       long fin = System.currentTimeMillis();

       // Calcula el tiempo total de ejecución
       long tiempoTotal = fin - inicio;

       System.out.println("El método tardó " + tiempoTotal +
               " milisegundos en ejecutarse para " + cantidad + " elementos.");
   }
}
```

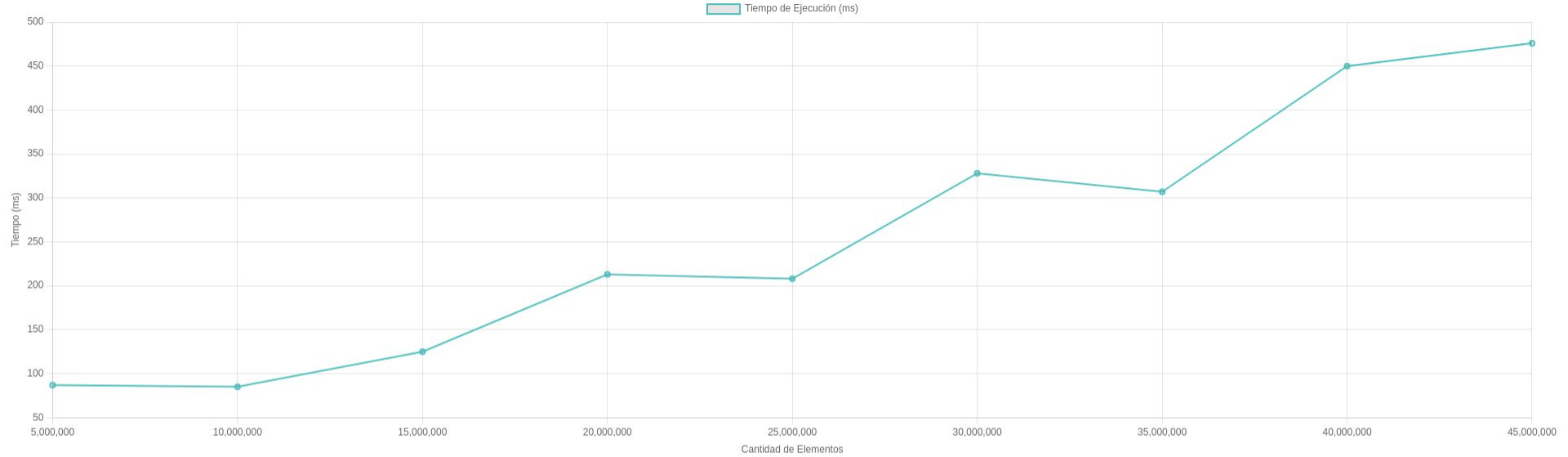
Tiempo de Ejecución

Propuesta 1: Calculemos el tiempo, en milisegundos, que tarda una operación en resolverse



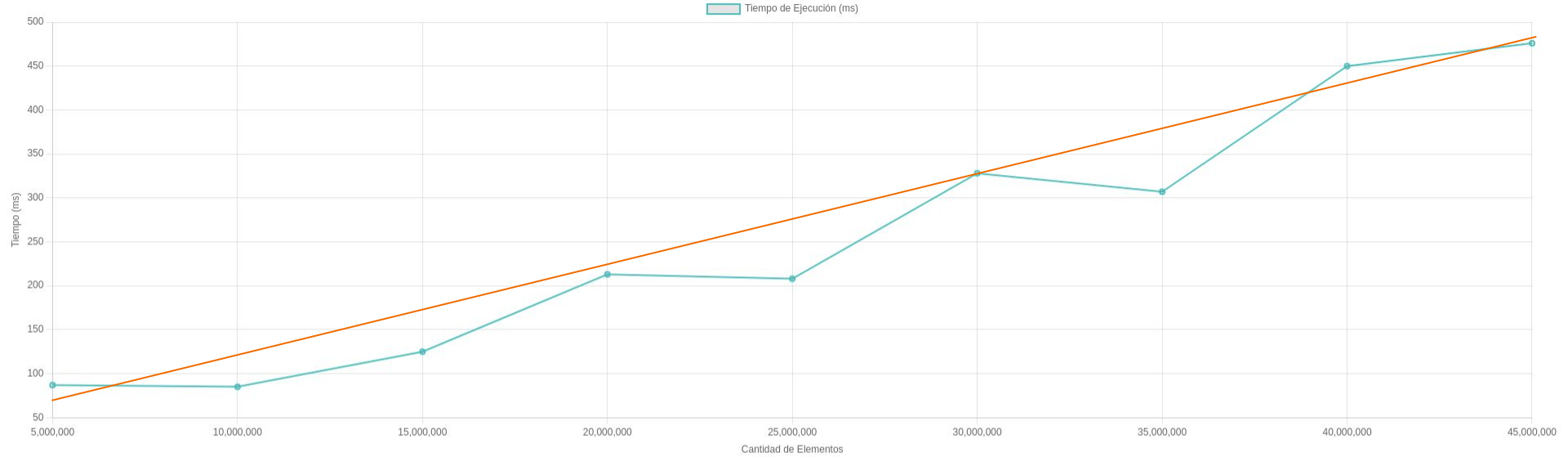
Tiempo de Ejecución

Propuesta 1: Calculemos el tiempo, en milisegundos, que tarda una operación en resolverse



Tiempo de Ejecución

Propuesta 1: Calculemos el tiempo, en milisegundos, que tarda una operación en resolverse



Tiempo de Ejecución

Propuesta 1: Calculemos el tiempo, en milisegundos, que tarda una operación en resolverse

¿Qué problema le vemos a esta propuesta?

Tiempo de Ejecución

Propuesta 1: Calculemos el tiempo, en milisegundos, que tarda una operación en resolverse

¿Qué problema le vemos a esta propuesta?

Los tiempos dependen del hardware
(procesador, memoria, disco, SO, compilador, etc)

No me sirve para algoritmos, sólo para implementaciones.

Tiempo de Ejecución

Como todo problema en la vida de la computación...

Tiempo de Ejecución

Como todo problema en la vida de la computación...

... lo podemos solucionar con abstracción.

ORDEN DEL TIEMPO DE EJECUCIÓN

Orden del Tiempo de Ejecución

Permite evaluar la eficiencia relativa de dos algoritmos independientemente del ambiente de hardware y software. Puede realizarse estudiando una versión de alto nivel del algoritmo sin necesidad de implementarlo o hacer experimentos.

Orden del Tiempo de Ejecución

El tiempo de ejecución de un algoritmo depende de la cantidad de operaciones primitivas realizadas.

Orden del Tiempo de Ejecución

Las operaciones primitivas toman tiempo constante y son:

– Asignar un valor a una variable

– Invocar un método

– Realizar una operación aritmética

– Comparar dos números

– Indexar un arreglo

– Seguir una referencia de objeto

– Retornar de un método

Orden del Tiempo de Ejecución

Como estos tiempos dependen del compilador y del hardware subyacente, por ello los notaremos con constantes arbitrarias c1, c2, c3, ...

Orden del Tiempo de Ejecución

Como estos tiempos dependen del compilador y del hardware subyacente, por ello los notaremos con constantes arbitrarias c1, c2, c3, ...

```
public static int concatenarYMultiplicar(int num1, int num2) {
    int resultado = num1 + num2;
    resultado = resultado * 10;
    resultado = resultado--;
    return resultado;
}
```

Orden del Tiempo de Ejecución

Como estos tiempos dependen del compilador y del hardware subyacente, por ello los notaremos con constantes arbitrarias c1, c2, c3, ...

```
public static int concatenarYMultiplicar(int num1, int num2) {
    int resultado = num1 + num2;     //c1
    resultado = resultado * 10;      //c2
    resultado = resultado--;         //c3
    return resultado;                //c4
} // c1 + c2 + c3 + c4 = c5
```

Orden del Tiempo de Ejecución

Si una instrucción es constante, decimos que es de Orden constante.

```
public static int concatenarYMultiplicar(int num1, int num2) {
    int resultado = num1 + num2;     //c1
    resultado = resultado * 10;      //c2
    resultado = resultado--;         //c3
    return resultado;                //c4
} // c1 + c2 + c3 + c4 = c5
```

Orden del Tiempo de Ejecución

Si una instrucción es constante, decimos que es de Orden constante.

Si un método tiene un tiempo constante, decimos que es de Orden constante.

```
public static int concatenarYMultiplicar(int num1, int num2) {
    int resultado = num1 + num2;     //c1
    resultado = resultado * 10;      //c2
    resultado = resultado--;         //c3
    return resultado;                //c4
} // c1 + c2 + c3 + c4 = c5
```

Orden del Tiempo de Ejecución

El método concatenarYMultiplicar es de Orden constante.

```
public static int concatenarYMultiplicar(int num1, int num2) {
    int resultado = num1 + num2;     //c1
    resultado = resultado * 10;      //c2
    resultado = resultado--;         //c3
    return resultado;                //c4
} // c1 + c2 + c3 + c4 = c5
```

Orden del Tiempo de Ejecución

El método concatenarYMultiplicar es de Orden constante. Esto quiere decir que el tiempo de ejecución de este método no es variable, no depende ni de los datos ingresados ni de las estructuras utilizadas.

```
public static int concatenarYMultiplicar(int num1, int num2) {
    int resultado = num1 + num2;     //c1
    resultado = resultado * 10;      //c2
    resultado = resultado--;         //c3
    return resultado;                //c4
} // c1 + c2 + c3 + c4 = c5
```

Orden del Tiempo de Ejecución

O(..) está O se la conoce como notación asintótica (Big-Oh)

Orden del Tiempo de Ejecución

O(..) está O se la conoce como notación asintótica (Big-Oh)

"f(n) es O(g(n))" se lee "f(n) es big-oh de g(n)" o "f(n) es del orden de g(n)"

Orden del Tiempo de Ejecución

O(..) está O se la conoce como notación asintótica (Big-Oh)

“f(n) es O(g(n))” se lee “f(n) es big-oh de g(n)” o “f(n) es del orden de g(n)”

concatenarYMultiplicar es O(1)

concatenarYMultiplicar es de orden constante

Orden del Tiempo de Ejecución

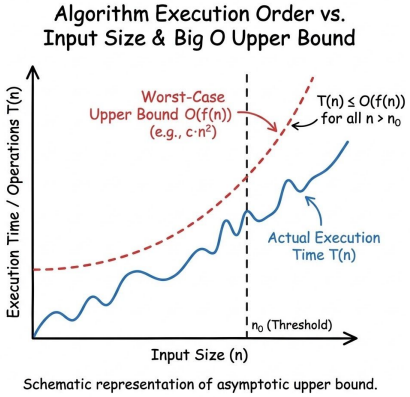
$T_A(n)$ es de O(f(n))

- Quiere decir que, el orden del tiempo de ejecución de un algoritmo A ejecutado sobre una entrada de tamaño n es del orden de f(n))
- Esto significa que el tiempo de corrida del peor caso puede ser acotado por arriba por f(n) multiplicado por un constante c a partir de un cierto tamaño de entrada n_0

Orden del Tiempo de Ejecución

$T_A(n)$ es de O(f(n))

- Quiere decir que, el orden del tiempo de ejecución de un algoritmo A ejecutado sobre una entrada de tamaño n es del orden de f(n))
- Esto significa que el tiempo de corrida del peor caso puede ser acotado por arriba por f(n) multiplicado por un constante c a partir de un cierto tamaño de entrada n_0



Schematic representation of asymptotic upper bound.

$T_A(n) \leq cf(n)$ para todo $n >= n_0$.

Es decir f es una cota superior del tiempo de A

t(n) y O(t(n))

Es importante distinguir la diferencia entre cada una:

-t(n) es la ecuación en términos de n (tamaño de la entrada) que obtenemos a partir del algoritmo.

-O(t(n)) es una cota superior de t(n) y determina el orden del tiempo de ejecución del algoritmo.

En caso de que el algoritmo tenga más de una entrada con distintos tamaños, la ecuación del tiempo de ejecución quedará expresada en más de una variable, por ej: t(n,m).

Reglas de la suma y el producto

Regla de la suma:

Si $f_1(n)$ es $O(g_1(n))$ y $f_2(n)$ es $O(g_2(n))$ entonces

$f_1(n) + f_2(n)$ es $O(\max(g_1(n), g_2(n))$

Regla del producto:

Si $f_1(n)$ es $O(g_1(n))$ y $f_2(n)$ es $O(g_2(n))$ entonces

$f_1(n) * f_2(n)$ es $O(g_1(n) * g_2(n))$

Ejercicio: Mostrar que $3n^2+2n+5$ es $O(n^2)$

Hay que hallar c real y n_0 natural tal que $3n^2+2n+5 \leq cn^2$ para $n \geq n_0$.
$3n^2 \leq 3n^2$
$2n \leq 2n^2$
$5 \leq 5n^2$
Luego $3n^2+2n+5 \leq 3n^2 + 2n^2 + 5n^2 = (3+2+5)n^2 = 10n^2$
Por lo tanto: c=10
Para hallar n_0 resolver $10n^2 \geq 3n^2+2n+5$,
lo que se hace resolviendo $7n^2-2n-5 \geq 0$
Con Bhaskara (o calculadora): $n \leq -5/7$ y $n \geq 1$
Descartamos la solución negativa
Luego $n_0=1$

Orden del Tiempo de Ejecución

¿Y ahora?

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0;
    for (Integer elemento : lista) {
        int resultado = concatenarYMultiplicar(elemento, 7);
        total = total + resultado;
    }
    return total;
}
```

Orden del Tiempo de Ejecución

¿Y ahora?

```java
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0; //c5
    for (Integer elemento : lista) {
        int resultado = concatenarYMultiplicar(elemento, 7);    //c6
        total = total + resultado;                              //c7
    }
    return total; // c8
}
```

Orden del Tiempo de Ejecución

¿Y ahora?

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0; //c5
    for (Integer elemento : lista) { //El cuerpo del foreach es c6 + c7 = c9
        int resultado = concatenarYMultiplicar(elemento, 7);    //c6
        total = total + resultado;                              //c7
    }
    return total; // c8
}
```

Orden del Tiempo de Ejecución

¿Cuántas veces se repite c9?

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0; //c5
    for (Integer elemento : lista) { //El cuerpo del foreach es c6 + c7 = c9
        int resultado = concatenarYMultiplicar(elemento, 7);    //c6
        total = total + resultado;                              //c7
    }
    return total; // c8
}
```

Orden del Tiempo de Ejecución

¿Cuántas veces se repite c9?

Tantas veces como elementos tenga la lista.

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0; //c5
    for (Integer elemento : lista) { //El cuerpo del foreach es c6 + c7 = c9
        int resultado = concatenarYMultiplicar(elemento, 7);    //c6
        total = total + resultado;                              //c7
    }
    return total; // c8
}
```

Orden del Tiempo de Ejecución

Sea *n* la cantidad de elementos que tiene la lista.

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0; //c5
    for (Integer elemento : lista) { //El cuerpo del foreach es c6 + c7 = c9
        int resultado = concatenarYMultiplicar(elemento, 7);    //c6
        total = total + resultado;                              //c7
    }
    return total; // c8
}
```

Orden del Tiempo de Ejecución

c5 +

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0; //c5
    for (Integer elemento : lista) { //El cuerpo del foreach es c6 + c7 = c9
        int resultado = concatenarYMultiplicar(elemento, 7);    //c6
        total = total + resultado;                              //c7
    }
    return total; // c8
}
```

Orden del Tiempo de Ejecución

c5 + n * c9 +

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0; //c5
    for (Integer elemento : lista) { //El cuerpo del foreach es c6 + c7 = c9
        int resultado = concatenarYMultiplicar(elemento, 7);    //c6
        total = total + resultado;                              //c7
    }
    return total; // c8
}
```

Orden del Tiempo de Ejecución

c5 + n * c9 + c8

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0; //c5
    for (Integer elemento : lista) { //El cuerpo del foreach es c6 + c7 = c9
        int resultado = concatenarYMultiplicar(elemento, 7);    //c6
        total = total + resultado;                              //c7
    }
    return total; // c8
}
```

Orden del Tiempo de Ejecución

c5 + n * c9 + c8 = el tiempo de ejecución

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0; //c5
    for (Integer elemento : lista) { //El cuerpo del foreach es c6 + c7 = c9
        int resultado = concatenarYMultiplicar(elemento, 7);    //c6
        total = total + resultado;                              //c7
    }
    return total; // c8
}
```

Orden del Tiempo de Ejecución

c5 + n * c9 + c8 = T(n) -> tiempo de ejecución

¿Cómo calculo el orden?

O(c) + O(n*c) + O(c) = máximo entre los órdenes (regla de la suma)

O(n*c) = c * O(n) = O(n) donde n es la cantidad de elementos de la lista

Orden del Tiempo de Ejecución

Podemos afirmar que concatenarYMultiplicarLista es de O(n) donde n es la cantidad de elementos de la lista.

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0;
    for (Integer elemento : lista) {
        int resultado = concatenarYMultiplicar(elemento, 7);
        total = total + resultado;
    }
    return total;
}
```

Orden del Tiempo de Ejecución

Esto significa que, mientras más elementos tenga la lista más va a tardar en resolver la operación de forma proporcional lineal.

```
public static int concatenarYMultiplicarLista(LinkedList<Integer> lista) {
    int total = 0;
    for (Integer elemento : lista) {
        int resultado = concatenarYMultiplicar(elemento, 7);
        total = total + resultado;
    }
    return total;
}
```

Orden del Tiempo de Ejecución

Esto significa que, mientras más elementos tenga la lista más va a tardar en resolver la operación de **forma proporcional lineal.**

Si se duplica la cantidad de elementos, estimamos que se va a duplicar el tiempo de resolución. Independientemente del hardware.

Funciones

- Constante: $f(n) = c$
- Logaritmo: $f(n) = \log_b(n)$ para $b>1$
- Lineal: $f(n) = n$
- N-LogN: $f(n) = n \log(n)$
- Cuadrática: $f(n) = n^2$
- Cúbica: $f(n) = n^3$
- Polinomial: $f(n) = n^k$, con k natural
- Exponencial: $f(n) = a^n$ con a real positivo
- Factorial: $f(n)=n!$

n	100	log2(n)	3n	nlog2(n)	n^2	n^3	2^n
10	100	3.321928	30	33.21928	100	1000	1024
20	100	4.321928	60	86.43856	400	8000	1048576
30	100	4.906891	90	147.2067	900	27000	1.07E+09
40	100	5.321928	120	212.8771	1600	64000	1.1E+12
50	100	5.643856	150	282.1928	2500	125000	1.13E+15
60	100	5.906891	180	354.4134	3600	216000	1.15E+18
70	100	6.129283	210	429.0498	4900	343000	1.18E+21
80	100	6.321928	240	505.7542	6400	512000	1.21E+24
90	100	6.491853	270	584.2668	8100	729000	1.24E+27
100	100	6.643856	300	664.3856	10000	1000000	1.27E+30
200	100	7.643856	600	1528.771	40000	8000000	1.61E+60
1000	100	9.965784	3000	9965.784	1000000	1E+09	1.1E+301
2000	100	10.96578	6000	21931.57	4000000	8E+09	#NUM!
10000	100	13.28771	30000	132877.1	1E+08	1E+12	#NUM!

Break

Reglas para calcular T(n) a partir del código fuente de un algoritmo

Paso 1: Determinar el tamaño de la entrada n

Paso 2: Aplicar las siguientes reglas en forma sistemática:

- $T_P(n)$ = constante si P es una acción primitiva
- $T_{S1;...;Sk}(n) = T_{S1}(n) + ... + T_{Sk}(n)$
- $T_{\text{if B then S1 else S2}}(n) = T_B(n) + \max(T_{S1}(n), T_{S2}(n))$
- $T_{for(m;S)}(n) = m*T_S(n)$ donde ***m***=cant_iteraciones y ***S*** el cuerpo del for
- $T_{\text{while B do S}}(n) = m*(T_B(n) + T_S(n)) + T_B(n)$ donde ***m***=cant_iteraciones (while ***B*** do ***S***)
- $T_{\text{call P}}(n) = T_S(n)$ donde ***procedure P***; y ***S*** el cuerpo del procedimiento
- $T_{f(e)}(n) = T_{x:=e}(n) + T_S(n)$ donde ***function f(x)***; y ***S*** el cuerpo de la función

Ejemplo: Calcular el orden del tiempo de ejecución de sumar la primera y la última componente de un arreglo a de n componentes

```
public int sumar_pri_ult(int [] a, int n) {
        int primera = a[0];
        int ultima = a[n-1];
        return primera + ultima;
}
```

Ejemplo: Calcular el orden del tiempo de ejecución de sumar la primera y la última componente de un arreglo a de n componentes

Entrada: arreglo a
Tamaño de la entrada: n = cantidad de elementos del arreglo

```
public int sumar_pri_ult(int [] a, int n) {
    int primera = a[0];
    int ultima = a[n-1];
    return primera + ultima;
}
```

c_1: porque [] y = son primitivas

c_2: idem

c_3: idem

Como es una secuencia, sumo los tiempos de las sentencias (o me quedo con el máximo de ellos).
$T(n) = c1 + c2 + c3 = c4 = c4 * 1$ para $n_0=0$
Entonces, $T(n) = O(1)$
Nota: Quiere decir que el tiempo que toma es independiente del tamaño del arreglo.

Ejemplo: Obtener orden de la búsqueda lineal en un arreglo de n enteros

```
public static int lsearch( int [] a, int n, int x ) {
        int i = 0;
        boolean encontre = false;
        while( i<n && !encontre )
                if( a[i] == x )
                        encontre = true;
                else i++;
        if( encontre ) return i;
        else return -1;
}
```

Ejemplo: Obtener orden de la búsqueda lineal en un arreglo de n enteros

```
public static int lsearch( int [] a, int n, int x ) {
        int i = 0;
        boolean encontre = false;
        while( i<n && !encontre )
                if( a[i] == x )
                        encontre = true;
                else i++;
        if( encontre ) return i;
        else return -1;
}
```

Tamaño de la entrada: n = cantidad de elementos del arreglo

c_1

c_2

Peor caso: n iteraciones - c_3 : condición del while

c_4: cuerpo

c_5 : tiempo del if después del while

$T(n) = c_1 + c_2 + (n(c_3+c_4) + c_3) + c_5 = O(n)$ - El tiempo es lineal en el tamaño de la entrada

Ejemplo: Obtener orden de la búsqueda lineal en un arreglo de n enteros (no estructurado)

```
public static int lsearch( int [] a, int n, int x ) {
        for ( int i = 0; i<n; i++ )
                if( a[i] == x )
                        return i;
        return -1;
}
```

¿Es más eficiente que el código anterior?

Ejemplo: Obtener orden de la búsqueda lineal en un arreglo de n enteros (no estructurado)

Tamaño de la entrada: n = cantidad de elementos del arreglo

```
public static int lsearch( int [] a, int n, int x ) {

    for ( int i = 0; i<n; i++ )

        if( a[i] == x )

            return i;

    return -1;

}
```

Peor caso: n iteraciones

c_1 : condición

c_2: cuerpo

c_3 :

$T(n) = n(c_1+c_2) + c_3 = O(n)$ - El tiempo es también lineal en el tamaño de la entrada para el peor caso. Es más eficiente porque realiza menos comparaciones que el anterior

Ejercicio: Obtener orden de la búsqueda binaria (dicotómica) en un arreglo de n componentes ordenadas en forma ascendente

```
public static int bsearch( int [] a, int n, int x ) {
        int ini = 0, fin = n-1;
        while( ini <= fin ) {
                int medio = (ini + fin) / 2;
                if( a[medio] == x ) return medio;
                else if( a[medio] > x ) fin = medio-1;
                else ini = medio + 1;
        }
return -1;
}
```

Ejercicio: Obtener orden de la búsqueda binaria (dicotómica) en un arreglo de n componentes ordenadas en forma ascendente

```
public static int bsearch( int [] a, int n, int x ) {
        int ini = 0, fin = n-1;
        while( ini <= fin ) {
                int medio = (ini + fin) / 2;
                if( a[medio] == x ) return medio;
                else if( a[medio] > x ) fin = medio-1;
                else ini = medio + 1;
        }
return -1;
}
```

Tamaño de la entrada: n = cantidad de elementos del arreglo

Peor caso: x no está en a

c_1

c_2: tiempo de condición

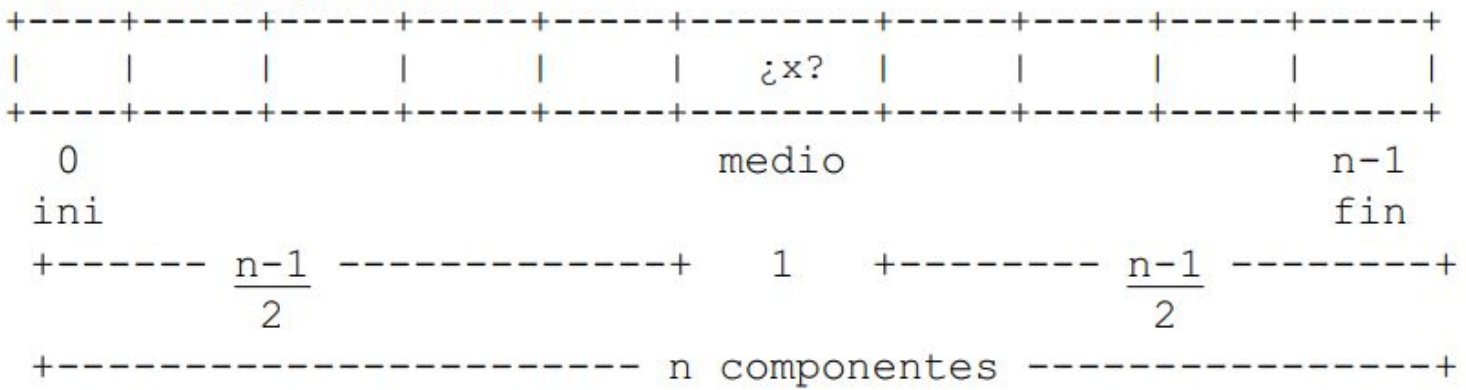
c_3 : tiempo del cuerpo

c_4 : tiempo del return

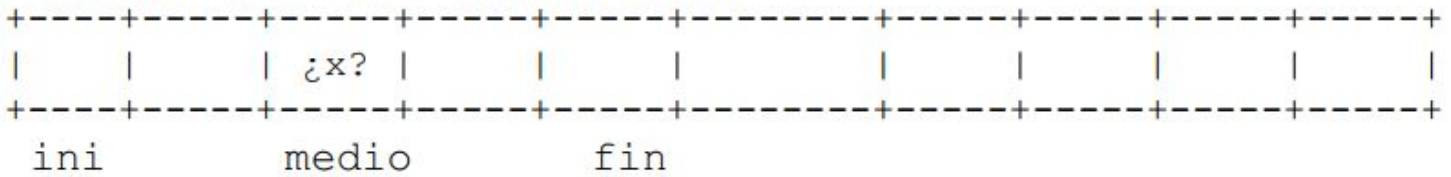
Sea k = cantidad de iteraciones del while, entonces

$T(n) = c_1 + k(c_2+c_3) + c_2 + c_4$. La pregunta es cómo definir k en función de n.

Al partir el arreglo por la mitad ocurre esto:



Al partir de nuevo en la segunda iteración:



+---------+ ¿Cuánto mide la mitad de la mitad menos 1?

$$\frac{\frac{(n-1)}{2} - 1}{2} = \frac{\frac{(n-1) - 2}{2}}{2} = \frac{n - 1 - 2}{4} = \frac{n-3}{4}$$

Número de iteración	Componentes por revisar
1	n
2	(n-1)/2
3	(n-3)/4
4	(n-7)/8
5	(n-15)/16
…	…
k	$\frac{n-(2^{k-1}-1)}{2^{k-1}}$

Entonces vimos que en la iteración k, la cantidad de componentes que quedan por revisar es:

$$\frac{n-(2^{k-1}-1)}{2^{k-1}}$$

Como x no está en el arreglo a, en la última iteración completa que realiza el while queda una componente del arreglo por revisar. Entonces:

$$\frac{n-(2^{k-1}-1)}{2^{k-1}}=1; \qquad n-(2^{k-1}-1)=2^{k-1};$$

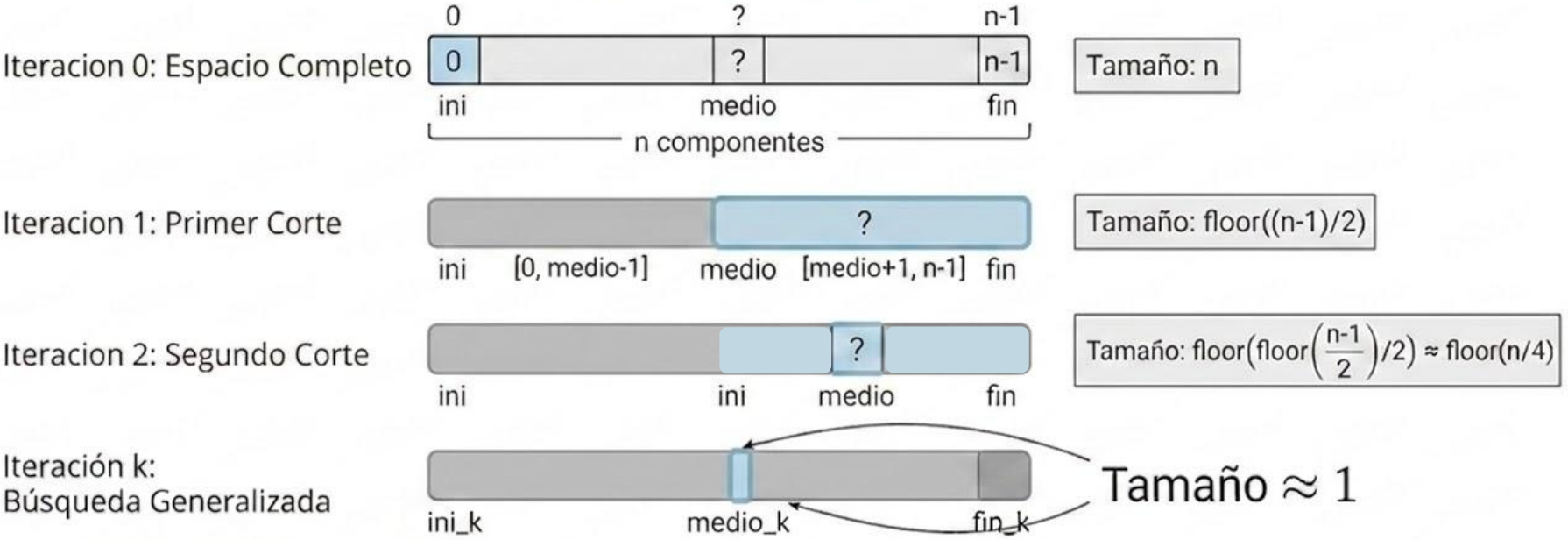
$$n-2^{k-1}+1=2^{k-1}; \qquad n+1=2^{k-1}+2^{k-1};$$

$$n+1=2\times 2^{k-1}; \qquad n+1=2^{k}; \qquad \log_2(n+1)=k.$$

Con lo que T(n) = c_1 + $\log_2(n+1)(c_2+c_3)$ + c_2 + c_4.
Luego, por regla de la suma, T(n) = O($\log_2(n)$).

Estimación de k: Visualización del Proceso y Generalización

Análisis de la reducción del espacio de búsqueda



Cálculo de k para el Peor Caso (Tamaño 1)

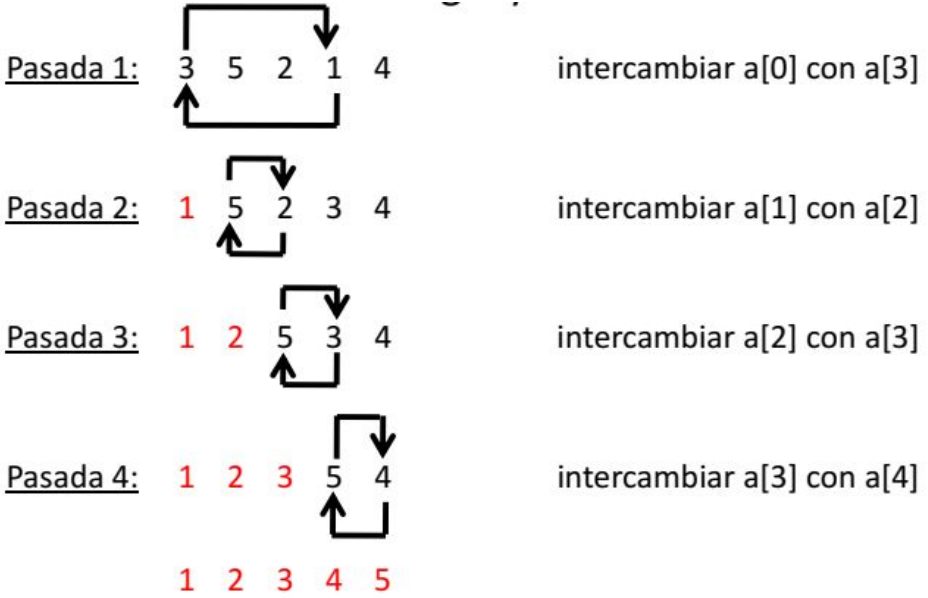
El espacio se reduce a 1 elemento cuando: $\frac{n}{2^k} = 1$ $2^k = n$ $k = \log_2(n)$

Reflexionemos

- Un int en Java mide 4 bytes
- Un arreglo de 1 terabyte tendría 1024*1024*1024*1024 bytes / 4 bytes = 274.877.906.944 componentes
- El logaritmo en base 2 de eso es 38
  - El while hace 38 iteraciones en el peor caso
  - La búsqueda lineal haría 274.877.906.944 iteraciones en el peor caso.

Ordenamiento por selección

- Haremos n-1 pasadas sobre el arreglo
- En la pasada i-ésima, encontramos el i-ésimo menor elemento del arreglo
  - y lo intercambiamos con a[i]



```
public static void selection_sort( int []a, int n ) {
        for( int i=0; i<n-1; i++ ) { // Repetir n-1 veces
                int k = i; // Hallar k tq a_k es el mínimo de
                for( int j=i+1; j<n; j++ ) // a_i, a_{i+1},..., a_{n-1}
                        if( a[j] < a[k] )
                                k = j;
                swap( a, i, k ); // Intercambiar a_i con a_k
        }
}
```

Tamaño del problema: n = cantidad de elementos del arreglo

```
public static void selection_sort( int []a, int n ) {
        for( int i=0; i<n-1; i++ ) {                    n-1 iteraciones
                int k = i;                              c1
                for( int j=i+1; j<n; j++ )              a lo sumo n-1 iteraciones
                        if( a[j] < a[k] )               c2
                                k = j;                  c3
                swap( a, i, k );                        c4
        }
}
```

$T(n) = (n-1).(c_1 + (n-1)(c_2 + c_3)+c_4)) = (n-1)c_1 + (n-1)^2 c_5 + (n-1)c_4$

$= (c_1+c_4)(n-1) + (n-1)^2 c_5 <= c_6(n-1)^2 <= c_7 n^2$

Po lo tanto, T(n) es O(n2)

Tamaño del problema: n = cantidad de elementos del arreglo

```
public static void selection_sort( int []a, int n ) {
        for( int i=0; i<n-1; i++ ) {                    n-1 iteraciones
                int k = i;                              c1
                for( int j=i+1; j<n; j++ )              comienza en i+1. Itera (n-1)-(i+1)+1
                        if( a[j] < a[k] )               c2
                                k = j;                  c3
                swap( a, i, k );                        c4
        }
}
```

Usando $\sum_{i=1}^{n} i = 1+2+\ldots+n = \frac{n(n+1)}{2}$

Se puede probar que es igualmente $O(n_2)$

Resultados

Si tengo un arreglo de n componentes:

- La búsqueda lineal tiene O(n)
- La búsqueda binaria tiene O(log(n))
- Los métodos de ordenamiento multipasada (selección, burbuja, inserción) tienen $O(n^2)$.
- Ejemplo: Si el arreglo mide n=1000 componentes:
  - Buscar linealmente toma 1000 pasos.
  - Buscar binariamente toma $\log_2(1000)$ 10 pasos.
  - Ordenar el arreglo toma $2(n-1) + n(n-1)/2$ 500000 pasos.
- Conclusión: Si el arreglo está ordenado, conviene búsqueda binaria. Ordenar el arreglo es caro.

Orden del Tiempo de Ejecución

Goodrich & Tamassia. Data Structures and Algorithms.

Fourth Edition. Capítulo 4.

Demostraciones

Propiedad auxiliar

Si $a \leq b$ y $c \leq d$ entonces $a + c \leq b + d$

Demostración:

Si $a \leq b$ entonces $b-a \geq 0$

Si $c \leq d$ entonces $d-c \geq 0$

La suma de dos números mayores o iguales a 0 es mayor o igual a 0, por lo tanto:

$(b-a) + (d-c) \geq 0$

Entonces $b+d \geq a+c$.

Luego, $a+c \leq b+d$

Demostración regla de la suma

Regla de la suma: Si $f_1(n)$ es $O(g_1(n))$ y $f_2(n)$ es $O(g_2(n))$ entonces

$f_1(n) + f_2(n)$ es $O(\max(g_1(n),g_2(n))$

Demostración:

Si $f_1(n)$ es $O(g_1(n))$ entonces existen c_1 y n_1 tales que $f_1(n) \leq c_1 g_1(n)$ para todo $n \geq n_1$

Si $f_2(n)$ es $O(g_2(n))$ entonces existen c_2 y n_2 tales que $f_2(n) \leq c_2 g_2(n)$ para todo $n \geq n_2$

Sea $n_0=\max(n_1,n_2)$, entonces

$f_1(n) \leq c_1 \max(g_1(n),g_2(n))$ para $n \geq n_0$

$f_2(n) \leq c_2 \max(g_1(n),g_2(n))$ para $n \geq n_0$

Entonces

$f_1(n)+f_2(n) \leq (c_1+c_2) \max(g_1(n),g_2(n))$ para $n \geq n_0$.

Luego $f_1(n) + f_2(n)$ es $O(\max(g_1(n),g_2(n))$, q.e.d

Demostración regla del producto

Regla del producto: Si $f_1(n)$ es $O(g_1(n))$ y $f_2(n)$ es $O(g_2(n))$ entonces
$f_1(n) * f_2(n)$ es $O(g_1(n)*g_2(n))$

Demostración:
Si $f_1(n)$ es $O(g_1(n))$ entonces c_1 y n_1 tales que $f_1(n) \leq c_1 g_1(n)$ para todo $n \geq n_1$
Si $f_2(n)$ es $O(g_2(n))$ entonces existen c_2 y n_2 tales que $f_2(n) \leq c_2 g_2(n)$ para todo $n \geq n_2$

Sea n_0=max(n_1,n,), entonces
$f_1(n) \leq c_1 g_1(n)$ para $n \geq n_0$
$f_2(n) \leq c_2 g_2(n)$ para $n \geq n_0$

Entonces $f_1(n)*f_2(n) \leq (c_1\ g_1(n))(c_2 g_2(n)) =$
$(c_1 c_2)(g_1(n)\ g_2(n))$ para $n \geq n_0$.
Luego $f_1(n) * f_2(n)$ es $O(g_1(n)*g_2(n))$, q.e.d.

Tamaño de la entrada: n = cantidad de componentes de a.

```
public static void selection_sort( int []a, int n ) {
    for( int i=0; i<n-1; i++ ) {  // n-1 iteraciones (desiguales)
        int k = i; // c1
        for( int j=i+1; j<n; j++ ) // (n-1)-(i+1)+1) iteraciones
                if( a[j] < a[k] ) // Tiempo del if: c2
                        k = j;
        swap( a, i, k ); // Tiempo de swap: c3
    }
}
```

$$T(n) = \sum_{i=0}^{n-2} (c_1 + ((n-1) - (i+1) + 1)c_2 + c_3)$$

$$T(n) = \sum_{i=0}^{n-2} (c_1 + ((n-1) - (i+1) + 1)c_2 + c_3) =$$

$$= \sum_{i=0}^{n-2} c_1 + c_2 \sum_{i=0}^{n-2} (n - i - 1) + \sum_{i=0}^{n-2} c_3$$

$$= c_1(n - 2 - 0 + 1) + c_2 \sum_{i=0}^{n-2} (n - i - 1) + c_3(n - 2 - 0 + 1)$$

$$= c_1(n-1) + c_2 \sum_{i=0}^{n-2} (n - i - 1) + c_3(n-1)$$

Falta ver cuánto vale el término del medio.

$$\sum_{i=0}^{n-2}(n-i-1) =$$

$$=(n-1)+(n-2)+(n-3)+\cdots+(n-(n-2)-1) =$$

$$=(n-1)+(n-2)+(n-3)+\cdots+1 =$$

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

Entonces, sabíamos que:

$$T(n) = c_1(n-1) + c_2 \sum_{i=0}^{n-2}(n-i-1) + c_3(n-1) =$$

$$= c_1\,(n-1) + c_2\frac{n(n-1)}{2} + c_3(n-1) = O(n^2)$$

THE END